

BRAIN TUMOR DETECTION USING CONVOLUTIONAL NEURAL NETWORKS

MRI-Based Multi-Class Classification System

Developed By

Chinmaya Sajeevan

Domain

Deep Learning | Computer Vision | Healthcare AI

Technologies Used

Python | TensorFlow | Keras | OpenCV | Scikit-learn | Streamlit

Dataset Source

Public Brain MRI Dataset (Kaggle)

Year

2026

Project Overview

This project focuses on automatic brain tumor classification from MRI images using a Convolutional Neural Network (CNN) built with TensorFlow and Keras.

The system classifies MRI scans into four categories:

- Glioma
- Meningioma
- Pituitary
- No Tumor

The model is trained on a public dataset from Kaggle and deployed using Streamlit for real-time prediction.

Problem Statement

Brain tumor diagnosis using MRI scans is a time-consuming and expertise-dependent task. Manual analysis may lead to:

- Human error
- Delayed diagnosis
- Inconsistent interpretation

This project aims to build an AI-based solution that:

- Automatically classifies tumor types
 - Assists medical professionals
 - Reduces diagnosis time
 - Improves prediction consistency
-

Dataset Information

- Source: Kaggle Brain Tumor MRI Dataset
- Categories:
 - Glioma
 - Meningioma
 - Pituitary

- No Tumor
 - Image Type: MRI scans (.jpg, .png)
-

Technologies Used

- Python
 - NumPy
 - OpenCV
 - Matplotlib & Seaborn
 - Scikit-learn
 - TensorFlow / Keras
 - Streamlit
-

Model Architecture

The CNN model consists of:

1 Convolution Layers

- Conv2D (32 filters, 3x3, ReLU)
- Conv2D (64 filters, 3x3, ReLU)
- Conv2D (128 filters, 3x3, ReLU)

Purpose:

- Extract spatial features like edges, shapes, textures.

2 MaxPooling Layers

- Reduces spatial dimensions
- Prevents overfitting
- Keeps important features

3 Flatten Layer

- Converts 2D feature maps into 1D vector

4 Fully Connected Layers

- Dense (128 neurons, ReLU)
- Dropout (0.5) to reduce overfitting
- Output layer (Softmax activation)

5 Output Layer

- 4 neurons (one per tumor class)
- Softmax activation for multi-class classification

Data Preprocessing

- ✓ Image resizing to 128×128
- ✓ Pixel normalization (0–1 scaling)
- ✓ Label Encoding
- ✓ One-Hot Encoding
- ✓ Train-Test Split (80-20)

Model Performance

- Loss Function: Categorical Crossentropy
- Optimizer: Adam
- Epochs: 20
- Batch Size: 16

Evaluation Metrics:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

The model achieved strong classification accuracy across tumor categories.

Confusion Matrix & Classification Report

- Confusion matrix visualized using Seaborn heatmap
 - Performance analyzed per tumor class
 - Helps identify misclassification patterns
-

Streamlit Deployment

An interactive web application was built using Streamlit.

Features:

- Upload MRI image
- Real-time prediction
- Confidence score display
- Clean medical-style interface

Run the app:

```
streamlit run app.py
```

Project Workflow

1. Load MRI dataset
 2. Preprocess images
 3. Encode labels
 4. Build CNN model
 5. Train model
 6. Evaluate performance
 7. Save model (.h5)
 8. Deploy with Streamlit
-

Skills Demonstrated

- Computer Vision
- Deep Learning

- CNN Architecture Design
- Model Evaluation
- Model Deployment
- Python Development
- UI Integration with Streamlit

THE CODE SNIPPETS

Prediction.ipynb

```
Import required libraries
```

```
#pip install opencv-contrib-python
```

```
import warnings
warnings.filterwarnings("ignore")
```

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPool2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
```

```
Load Dataset Path
```

```
data_path="E:\Cancer_Detection_Dataset_PROJECT\dataset\Training"
```

Read and Preprocess Images

```
x = []
y = []

for category in os.listdir(data_path):
    category_path = os.path.join(data_path, category)

    if os.path.isdir(category_path):
        for file in os.listdir(category_path):
            img_path = os.path.join(category_path, file)

            if img_path.endswith(('.jpg', '.png', '.jpeg')):
                img = cv2.imread(img_path)
                img = cv2.resize(img, (128, 128))
                img = img / 255.0

                x.append(img)
                y.append(category)

x = np.array(x)
y = np.array(y)

print("Images shape:", x.shape)
print("Labels shape:", y.shape)
```

```
Images shape: (5600, 128, 128, 3)
Labels shape: (5600,)
```

Encode Labels

```
le = LabelEncoder()
y = le.fit_transform(y)
y = to_categorical(y)
```

Train-Test Split

```
xtrain, xtest, ytrain, ytest = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

```
print(len(xtrain),len(ytrain))  
print(len(xtest),len(ytest))
```

```
4480 4480  
1120 1120
```

Build CNN Model

```
model = Sequential()  
  
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))  
model.add(MaxPool2D(2,2))  
  
model.add(Conv2D(64, (3,3), activation='relu'))  
model.add(MaxPool2D(2,2))  
  
model.add(Conv2D(128, (3,3), activation='relu'))  
model.add(MaxPool2D(2,2))  
  
model.add(Flatten())  
  
model.add(Dense(128, activation='relu'))  
model.add(Dropout(0.5))  
  
model.add(Dense(len(le.classes_), activation='softmax'))
```

Compile Model

```
model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```


Train Model

```
history = model.fit(  
    xtrain, ytrain,  
    epochs=20,  
    batch_size=16,  
    validation_data=(xtest, ytest)  
)
```

```
# Evaluate model on test data  
test_loss, test_accuracy = model.evaluate(xtest, ytest)  
  
print("Test Loss:", test_loss)  
print("Test Accuracy:", test_accuracy)  
print("Test Accuracy (%)ate", test_accuracy * 100)
```

35/35 ————— 3s 70ms/step - accuracy: 0.9473 - loss: 0.2812
Test Loss: 0.2812063694000244
Test Accuracy: 0.9473214149475098
Test Accuracy (%): 94.73214149475098

```
# Predict on test set  
y_pred = model.predict(xtest)  
  
# Convert one-hot to class index  
y_pred_classes = np.argmax(y_pred, axis=1)  
y_true = np.argmax(ytest, axis=1)  
  
# Create confusion matrix  
cm = confusion_matrix(y_true, y_pred_classes)  
  
print("Confusion Matrix:\n", cm)
```

35/35 ————— 3s 69ms/step
Confusion Matrix:
[[296 4 0 0]
 [29 251 8 0]
 [2 11 244 0]
 [1 4 0 270]]

```

from sklearn.metrics import classification_report

print("\nClassification Report:\n")
print(classification_report(y_true, y_pred_classes, target_names=le.classes_))

```

Classification Report:

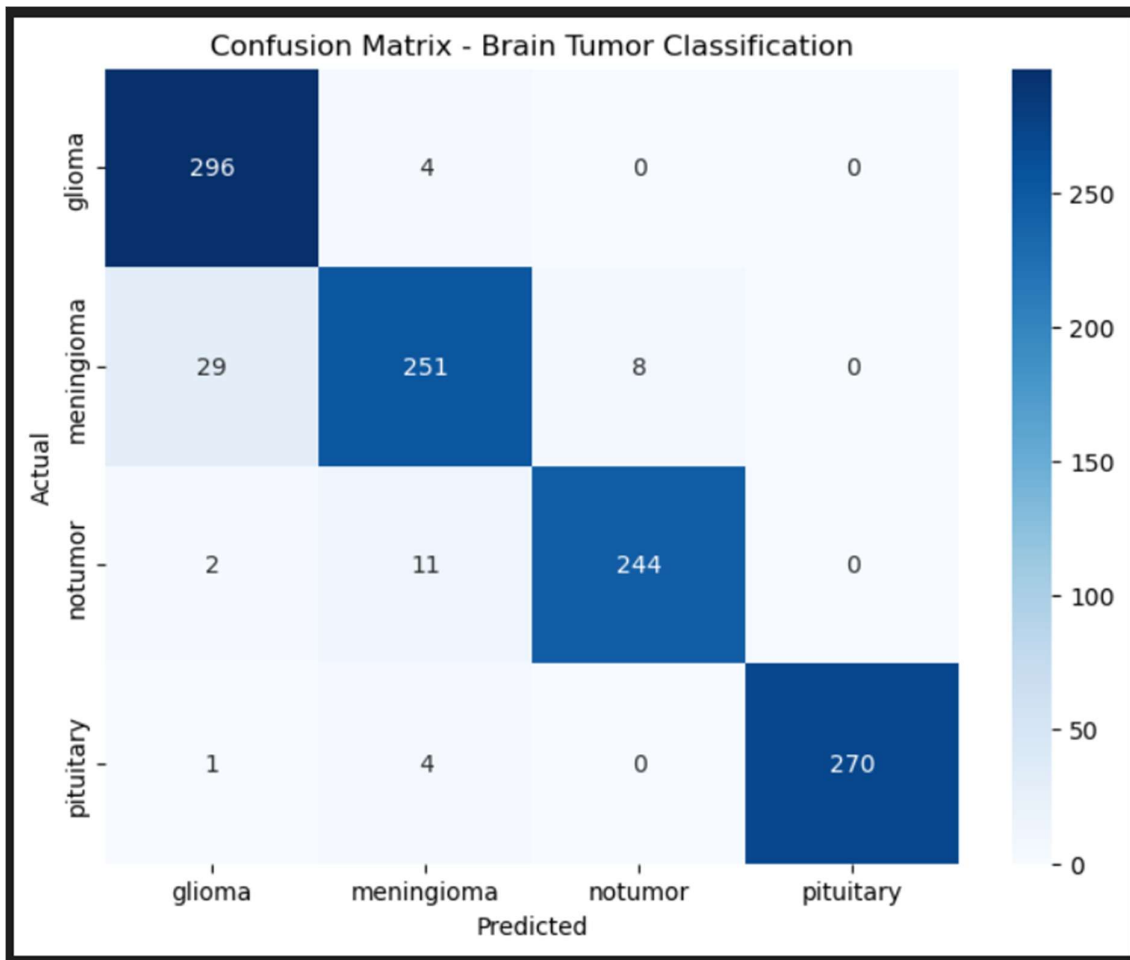
	precision	recall	f1-score	support
glioma	0.90	0.99	0.94	300
meningioma	0.93	0.87	0.90	288
notumor	0.97	0.95	0.96	257
pituitary	1.00	0.98	0.99	275
accuracy			0.95	1120
macro avg	0.95	0.95	0.95	1120
weighted avg	0.95	0.95	0.95	1120

```

plt.figure(figsize=(8,6))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            xticklabels=le.classes_,
            yticklabels=le.classes_)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Brain Tumor Classification")
plt.show()

```



Save Model

```
model.save("brain_tumor_model.h5")  
print("Model Saved Successfully!")
```

```
test_data1="E:\\Cancer_Detection_Dataset_PROJECT\\dataset\\Testing\\meningioma\\Te-aug-me_2.jpg"
```

```
from tensorflow.keras.models import load_model
```

```
# Load model
```

```
model = load_model("brain_tumor_model.h5")
```

```
img=cv2.imread(test_data1)           # Read Image
img = cv2.resize(img, (128,128))     # Resize to model input size
img = img / 255.0                     # Normalize
img = img.reshape(1,128,128,3)       # Add batch dimension
```

```
prediction=model.predict(img)
prediction
```

```
1/1 ————— 0s 150ms/step
```

```
array([[1.6773477e-01, 8.3224273e-01, 2.2485781e-05, 5.0363597e-10]],
      dtype=float32)
```

```
#Get class index
pred_class=np.argmax(prediction)

#Convert to class name
predicted_label=le.inverse_transform([pred_class])[0]

print("Predicted Tumor Type:",predicted_label)
```

```
Predicted Tumor Type: meningioma
```

App.py

```
import streamlit as st
import numpy as np
import cv2
from tensorflow.keras.models import load_model
from PIL import Image

# -----
# Load Trained Model
# -----
model = load_model("brain_tumor_model.h5")

# Class names (must match training folder names exactly)
labels = ['glioma', 'meningioma', 'notumor', 'pituitary']

# -----
# Streamlit UI
# -----
st.title("Brain Tumor Detection using CNN")
st.write("Upload an MRI image to predict the tumor type.")

uploaded_file = st.file_uploader("Choose an MRI Image", type=["jpg", "jpeg", "png"])
```

```

if uploaded_file is not None:

    # Convert uploaded file to RGB
    image = Image.open(uploaded_file).convert("RGB")
    st.image(image, caption="Uploaded Image", use_column_width=True)

    # Convert to numpy
    img = np.array(image)

    # Resize
    img = cv2.resize(img, (128, 128))

    # Normalize
    img = img / 255.0

    # Add batch dimension properly
    img = np.expand_dims(img, axis=0)

    # Prediction
    prediction = model.predict(img)

    pred_class = np.argmax(prediction)
    predicted_label = labels[pred_class]
    confidence = np.max(prediction) * 100

    st.subheader("Prediction Result:")
    st.success(f"Predicted Tumor Type: {predicted_label}")
    st.info(f"Confidence: {confidence:.2f}%")

```

The Output

Brain Tumor Detection using CNN

Upload an MRI image to predict the tumor type.

Choose an MRI Image



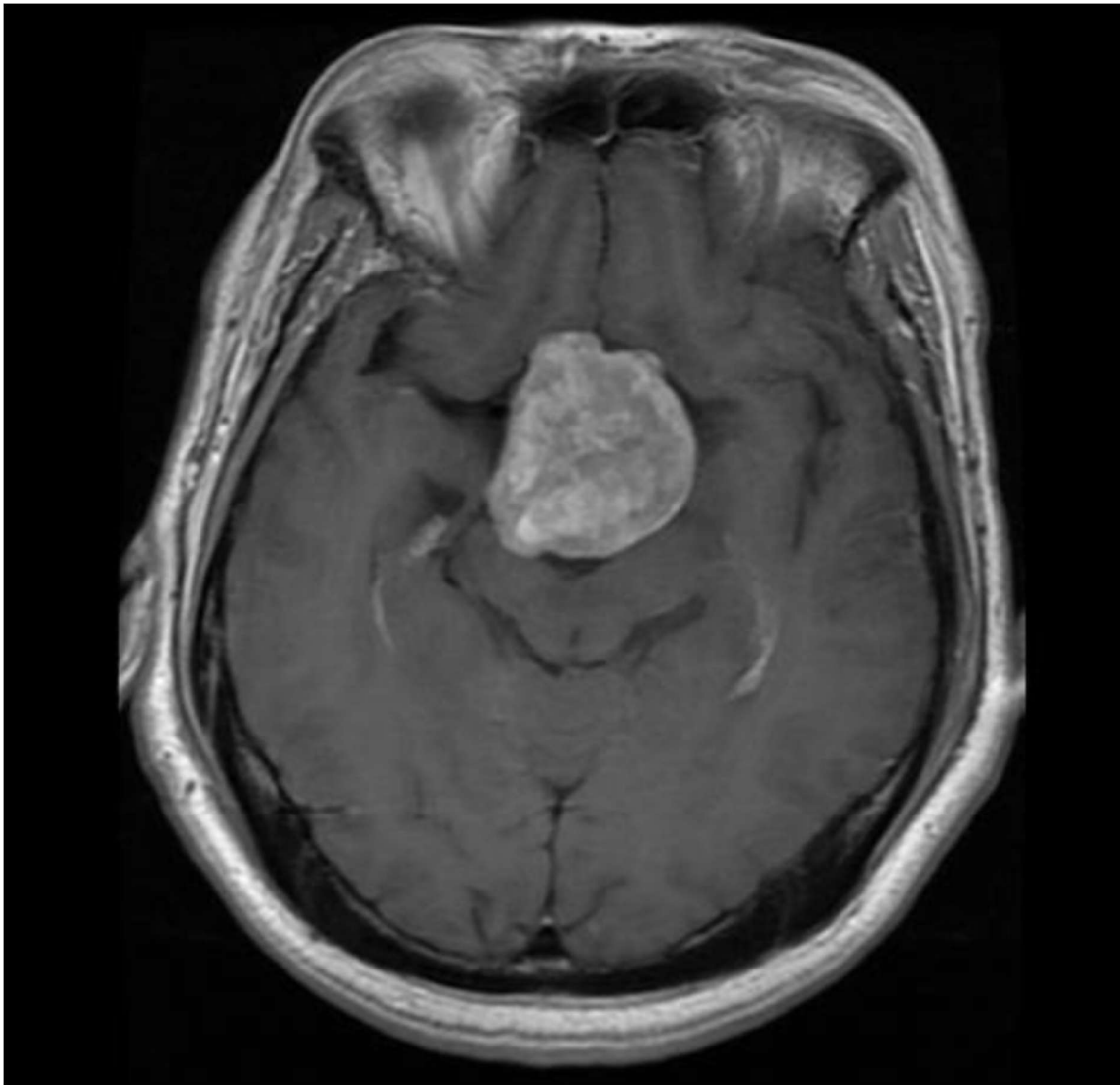
Drag and drop file here

Limit 200MB per file • JPG, JPEG, PNG

Browse files



Te-pi_2.jpg 26.3KB



Prediction Result:

Predicted Tumor Type: pituitary

Confidence: 100.00%

Conclusion

This project successfully developed a Convolutional Neural Network (CNN) model for multi-class brain tumor classification using MRI images. The model was trained on a publicly available dataset from Kaggle and demonstrated strong classification performance across Glioma, Meningioma, Pituitary, and No Tumor categories.

Through effective preprocessing, model design, and evaluation using accuracy metrics and confusion matrix analysis, the system achieved reliable prediction capability. The trained model was further deployed using Streamlit to enable real-time tumor classification from uploaded MRI scans.

This project demonstrates practical application of deep learning in medical image analysis and highlights the potential of AI-assisted diagnostic systems in healthcare.

Future improvements may include transfer learning, data augmentation, and model explainability techniques such as Grad-CAM to enhance performance and interpretability.